Superstore Data Pipeline

Technical Documentation

An ELT pipeline with Apache Airflow, dbt and DuckDB

STUDENT

Omar Mohamed Hosny

INSTITUTION

NTI - National Telecommunication Institute

INSTRUCTOR

Kyrellos El Naggar

DATE

3 October 2026

SOURCE CODE

https://github.com/omarmohosny/superstore-pipeline

Table of Contents

1. Introduction **5**

1.1 Purpose . . . 5
1.2 The dataset . . . 5
1.3 Goals . . . 5
1.4 Scope and out-of-scope items . . . 5

2. Technology Stack **5**

3. System Architecture **6**

3.1 Overview . . . 6
3.2 Containers . . . 6
3.3 Repository layout . . . 7

4. Data Layers **7**

4.1 Medallion design . . . 7
4.2 Bronze: incremental loading . . . 8
4.3 Silver: validation and typing . . . 9
4.4 Gold: star schema . . . 9
4.5 Marts . . . 9

5. Orchestration with Airflow **10**

5.1 The DAG . . . 10
5.2 Quality gate . . . 10
5.3 Failure handling and run log . . . 11

6. Data Model **11**

6.1 Star schema . . . 11
6.2 Data dictionary . . . 12

7. Testing and Data Quality **14**

7.1 Approach . . . 14
7.2 Results . . . 14
7.3 Why reconciliation tests matter . . . 14

8. Deployment and Operation **14**

8.1 Prerequisites . . . 14
8.2 Start the stack . . . 15
8.3 Useful commands . . . 15
8.4 Loading corrected data . . . 15

9. Results **15**

9.1 Business results from the marts 15
9.2 Pipeline evidence 16

10. Issues Found and Fixed **18**

Known limitations 18

11. Future Work **18**

12. Conclusion **19**

Appendix A: Key Source Code **20**

A.1 Reject rules (dbt model silver_checked) 20
A.2 Incremental fact table (dbt model fact_sales) 20
A.3 Bronze loader core (scripts/load_bronze.py) 20
A.4 DAG definition (airflow/dags/superstore_pipeline.py) 21

List of Figures

Figure 1: System architecture 6
Figure 2: Docker Compose services and bind mounts 7
Figure 3: Data flow from the Excel file to the marts 8
Figure 4: The superstore_pipeline DAG 10
Figure 5: Star schema of the gold layer 11
Figure 6: Monthly sales and profit, 2014 to 2017 (mart_monthly_sales) 15
Figure 7: Sales and profit by category, and customers per RFM segment 16
Figure 8: Average days from order to shipment by ship mode 16
Figure 9: Airflow graph view of the DAG with all tasks successful 16
Figure 10: Airflow run history with successful and failed runs 17
Figure 11: Log of the test_silver quality gate 17
Figure 12: The five containers of the stack in Docker Desktop 17
Figure 13: dbt lineage graph from the generated documentation 17

List of Tables

Table 1: Technologies used 6
Table 2: Layers and their objects 8
Table 3: Reject rules (evaluated in this order) 9
Table 4: Analytics marts 9
Table 5: DAG tasks 10
Table 6: Configuration 11
Table 7: bronze.raw_superstore (metadata columns; the other 21 columns mirror the source file in snake_case) 12
Table 8: gold.dim_customer 12
Table 9: gold.dim_product 12
Table 10: gold.dim_location 12
Table 11: gold.dim_date 12
Table 12: gold.fact_sales (grain: one order line) 13
Table 13: marts.* (columns of the four marts) 13
Table 14: ops.pipeline_run_log 13
Table 15: Automated checks 14
Table 16: Test results at the end of the project 14
Table 17: Defects and their fixes 18

1. Introduction

1.1 Purpose

This document describes the design, implementation, testing and operation of the **Superstore Data Pipeline**, an end-to-end ELT (extract, load, transform) project. The pipeline takes a spreadsheet of retail sales orders, loads it into an analytical database, validates and cleans it, models it as a star schema, and publishes ready-to-query reporting tables. Apache Airflow runs the whole process on a schedule inside Docker containers.

1.2 The dataset

The source is *Superstore.xlsx*, a sample retail dataset with a single sheet named *Sample - Superstore*. It contains 9,994 order lines and 21 columns that cover orders, shipping, customers, geography, products and the measures sales, quantity, discount and profit. Orders run from 03 January 2014 to 30 December 2017, across 5,009 orders, 793 customers and 1,862 products.

1.3 Goals

- Load the Excel file into a database and keep the raw data untouched, with load metadata for traceability.
- Clean and type the data, and quarantine invalid rows with a reason instead of silently dropping them.
- Model the clean data as a star schema and build analytics tables for monthly sales, profit, customer segments and shipping speed.
- Run everything on a schedule, stop automatically when data quality is poor, and record every run.
- Make the project reproducible: one command starts the whole stack, and automated tests protect its behaviour.

1.4 Scope and out-of-scope items

The project covers ingestion, transformation, orchestration, testing and local deployment. A dashboard layer, e-mail or chat alerting, CI/CD and cloud deployment are intentionally out of scope.

2. Technology Stack

Each tool was chosen for a clear job and kept as simple as the requirements allow.

Table 1: Technologies used

Tool	Version	Role in the project
Apache Airflow	3.1.0	Schedules and monitors the pipeline; one DAG with ten tasks, retries and failure callbacks.
dbt Core + dbt-duckdb	1.12.5 / 1.11.0	SQL transformations, tests, snapshots, documentation.
DuckDB	1.5.6	Embedded analytical database stored in a single file.
Python, pandas, openpyxl	3.12 (container)	Reads the Excel file and loads the bronze table; run logging and quality checks.
PostgreSQL	16	Airflow metadata database (not used for business data).
Docker Compose	v5	Runs Airflow, PostgreSQL and the tools as one reproducible stack.
pytest	-	Unit tests for the loader, the ops script and the DAG structure.
Git and GitHub	-	Version control and publication of the source code.

dbt and the loader are installed in a dedicated Python virtual environment inside the Airflow image. This keeps their dependencies separate from Airflow's pinned dependencies, so neither side can break the other.

3. System Architecture

3.1 Overview

The system has two parts that share one DuckDB file. The **control plane** is Airflow, which decides what runs and when. The **data plane** is a set of scripts and dbt commands that run as bash tasks and read or write the database. Figure 1 shows the components and how they connect.

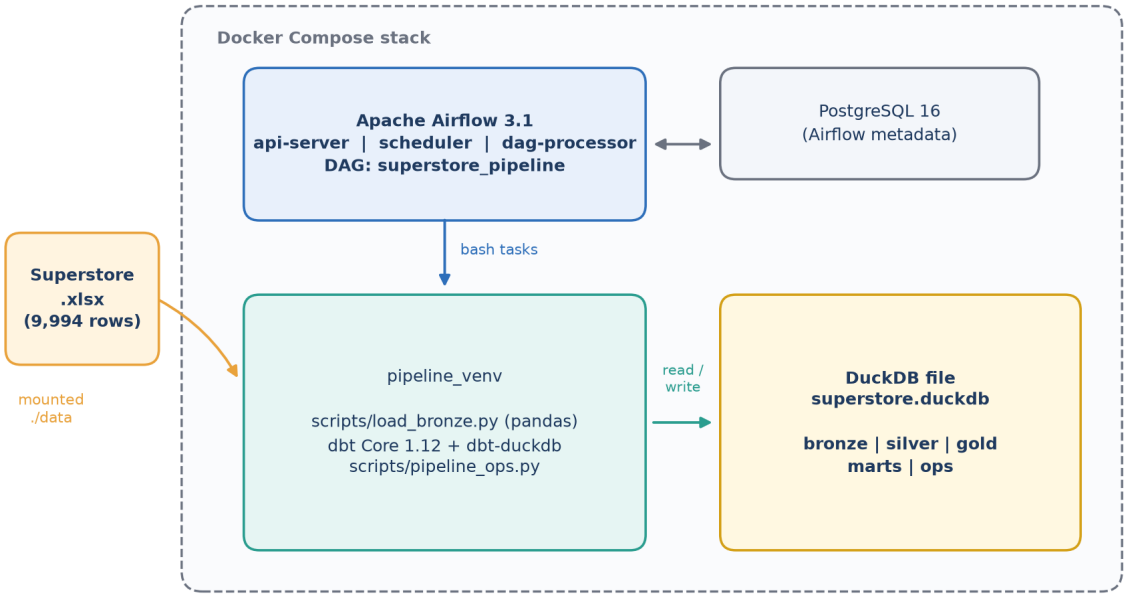


Figure 1: System architecture

3.2 Containers

Docker Compose starts five services. The init service migrates the Airflow metadata database once and exits; the other Airflow services wait for it. Every Airflow container mounts the same project folders, so a change to a model or script is visible immediately without rebuilding the image.

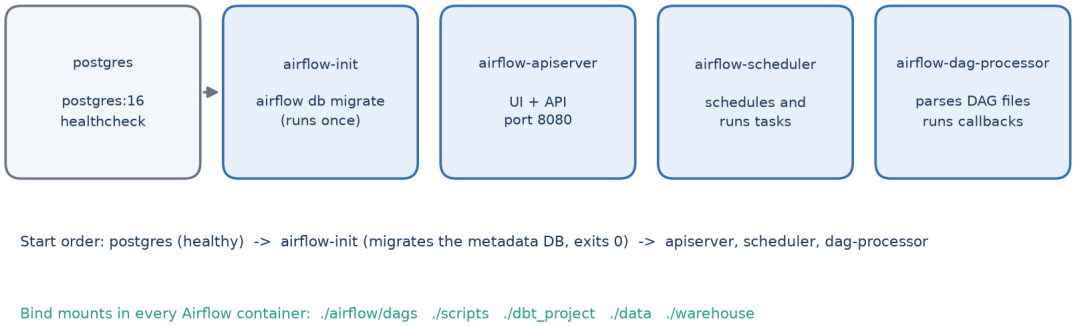


Figure 2: Docker Compose services and bind mounts

3.3 Repository layout

```
superstore_project/
  airflow/            Dockerfile, requirements.txt, docker-compose.yaml, dags/superstore_pipeline.py
  data/raw/           Superstore.xlsx (source file)
  scripts/            load_bronze.py, pipeline_ops.py, query.py, in_container.sh
  dbt_project/        dbt_project.yml, profiles.yml, macros/, snapshots/, tests/
    models/           bronze (sources), silver, gold, marts
  tests/              pytest: test_load_bronze.py, test_pipeline_ops.py, test_dag.py
  warehouse/          superstore.duckdb (created at run time, not in Git)
  docs/               specification, implementation plan, this document
```

4. Data Layers

4.1 Medallion design

The data moves through four layers, each in its own database schema. A layer only reads from the one before it, which keeps the flow easy to follow and to test.

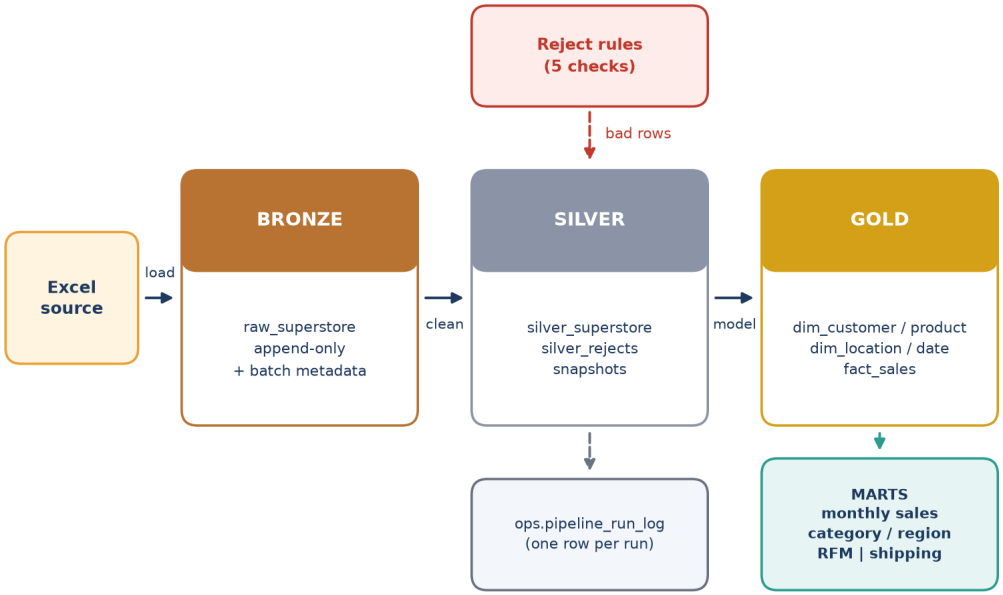


Figure 3: Data flow from the Excel file to the marts

Table 2: Layers and their objects

Schema	Objects	Materialization	Purpose
`bronze`	raw_superstore	table (Python)	Raw copy of the file with batch metadata. Append-only.
`silver`	silver_checked, silver_superstore, silver_rejects, silver_customers, silver_products, snap_customer, snap_product	view / table / snapshot	Typed, trimmed and validated rows; rejected rows with a reason; slowly changing history.
`gold`	dim_customer, dim_product, dim_location, dim_date, fact_sales	table / incremental	Star schema for analytics.
`marts`	mart_monthly_sales, mart_category_region_profit, mart_customer_rfm, mart_shipping_performance	table	Ready-to-use reporting tables.
`ops`	pipeline_run_log	table (Python)	One row per pipeline run.

4.2 Bronze: incremental loading

`scripts/load_bronze.py` reads the sheet with pandas and writes `bronze.raw_superstore`. Column names are only converted to snake_case; values are not cleaned, so bronze always shows what the source contained. Three metadata columns are added to every row: `_batch_id`, `_loaded_at` and `_source_file`.

- **Incremental:** only rows whose `row_id` is not already loaded are inserted. Running the loader twice on the same file adds zero rows, and a file with extra rows loads only the new ones.
- **Dates:** Excel often stores dates as serial numbers (for example 42682 is 8 November 2016). Numeric cells are converted from serial numbers; text is parsed; anything unreadable becomes null and is rejected later in silver instead of crashing the load.

- **Postal codes:** they are read as text, and numeric codes that lost their leading zero (449 rows, such as 6824) are padded back to five digits (06824).
- **Fail early:** a missing file, a missing sheet, missing columns, a header-only sheet, empty or duplicate `row_id` values all stop the load with a clear message.

4.3 Silver: validation and typing

The model `silver_checked` casts every column to its real type, trims text, and computes a `reject_reason` with five rules (Table 3). Two models then split the result: `silver_superstore` keeps the valid rows and `silver_rejects` keeps the others together with the reason. Nothing is deleted, so every bronze row ends up in exactly one of the two tables; a dbt reconciliation test enforces this.

Table 3: Reject rules (evaluated in this order)

Reason	Condition
`missing key`	row_id, order_id, customer_id or product_id is null
`missing date`	order_date or ship_date is null (or unreadable)
`non-positive sales`	sales is null or sales <= 0
`ship date before order date`	ship_date < order_date
`discount out of range`	discount < 0 or discount > 1

On the real dataset all 9,994 rows pass, so 0 rows are rejected. The rules matter for future files.

`silver_customers` and `silver_products` hold one row per customer and product, using the most recent attributes. A few product IDs appear with more than one name in the source, so the latest name is kept. Two dbt snapshots, `snap_customer` and `snap_product`, keep the history of those attributes using the *check* strategy (slowly changing dimension, type 2). Because the dataset is static, each key has a single version until the source changes.

4.4 Gold: star schema

The gold layer builds four dimensions and one fact table. Surrogate keys are the MD5 hash of the natural key, so they stay identical every time a dimension is rebuilt and the fact table never loses its links. `dim_date` is generated as a calendar that covers every order and ship date. `fact_sales` is an *incremental* model with `row_id` as unique key: on later runs it only processes rows loaded after the newest row already in the table.

4.5 Marts

Table 4: Analytics marts

Mart	Grain	What it answers
`mart_monthly_sales`	month	Sales, profit, orders and units per month.
`mart_category_region_profit`	category, sub-category, region	Where is the business profitable?
`mart_customer_rfm`	customer	Recency, frequency, monetary value, quartile scores and a segment label.
`mart_shipping_performance`	ship mode, region	Average and maximum days from order to shipment.

RFM scores use quartiles (1 to 4) of recency, frequency and monetary value. Recency is measured from the newest order in the data, not from today's date, so the result does not change as time passes. Segments are assigned in this order: **Champion** (R, F and M all 3 or higher), **Loyal** (R 3 or higher and F 2 or higher), **At risk** (R 2 or lower and F 3 or higher), **Lost** (R equal to 1), and **Regular** for the rest.

5. Orchestration with Airflow

5.1 The DAG

The DAG `superstore_pipeline` runs daily and has ten tasks in a straight line. Every task is a bash command, written with Airflow's TaskFlow `@task.bash` decorator. DuckDB allows only one writer at a time, so the tasks run one after another and only one DAG run is active at once.

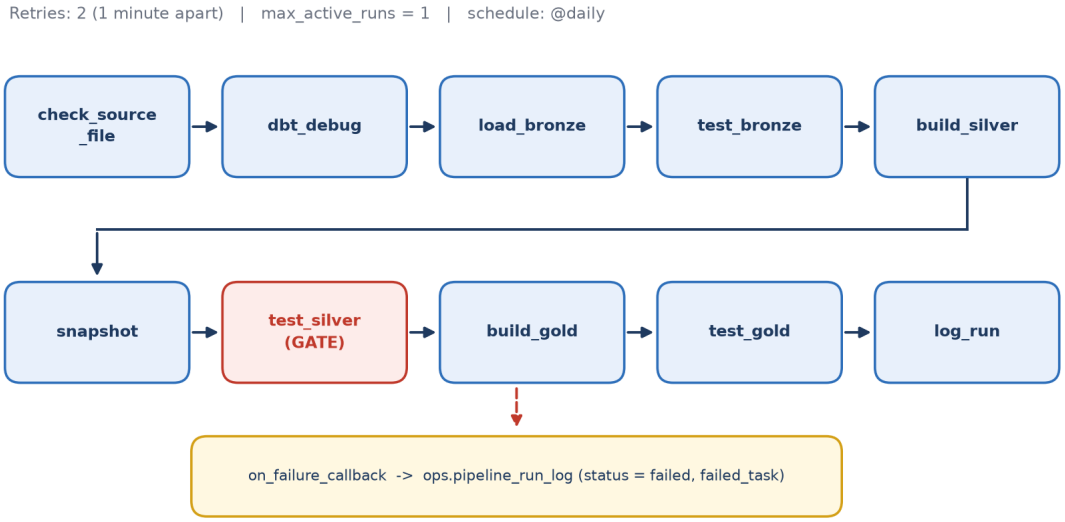


Figure 4: The superstore_pipeline DAG

Table 5: DAG tasks

Task	Command	Purpose
`check_source_file`	test -f $SUPERSTORE_FILE	Fail fast if the Excel file is missing.
`dbt_debug`	dbt debug	Check the dbt project and database connection.
`load_bronze`	load_bronze.py	Append new rows to bronze.
`test_bronze`	dbt test --select source:bronze	Not-null and unique checks on the raw table.
`build_silver`	dbt run --select silver	Type, validate and split the data.
`snapshot`	dbt snapshot	Record customer and product history.
`test_silver`	dbt test (silver) + check-rejects	Quality gate: dbt tests, then the reject-rate limit.
`build_gold`	dbt run --select gold marts	Build dimensions, fact table and marts.
`test_gold`	dbt test (gold, marts) + docs generate	Keys, relationships, reconciliation; refresh dbt docs.
`log_run`	pipeline_ops.py log	Write the success row to ops.pipeline_run_log.

5.2 Quality gate

The gold layer is built only if `test_silver` succeeds. It runs the silver dbt tests and then `pipeline_ops.py check-rejects`, which compares the rejected rows of the *current batch* with the

rows that batch loaded. If the rate is above the limit (2% by default), the task fails and the later tasks do not run. The limit can be changed with the Airflow Variable `reject_rate_threshold`, with no code change. A run that loads no new rows always passes this check.

5.3 Failure handling and run log

Each task retries twice, one minute apart. When a task finally fails, the `on_failure_callback` writes a row with status *failed* and the name of the failing task to `ops.pipeline_run_log`. The callback works even if no bronze or silver tables exist yet, for example when the source file is missing. Successful runs are logged by the last task. Each row records the batch id, status, rows loaded, rows rejected, duration, failed task and finish time.

Table 6: Configuration

Setting	Where	Meaning
`SUPERSTORE_FILE`	docker-compose.yaml	Path of the source Excel file inside the containers.
`SUPERSTORE_DB`	docker-compose.yaml	Path of the DuckDB file; also used by dbt through profiles.yml.
`PIPELINE_BIN`	Dockerfile	Folder of the dedicated virtual environment (dbt and python).
`reject_rate_threshold`	Airflow Variable	Maximum share of rejected rows per batch (default 0.02).
`schedule / retries`	DAG file	@daily, 2 retries with a 1-minute delay, max_active_runs = 1.

6. Data Model

6.1 Star schema

Figure 5 shows the gold layer. One fact table at order-line grain (9,994 rows) references four dimensions (793 customers, 1,862 products, 632 locations and 1,464 calendar days). All relationships are many-to-one and are verified by dbt tests.

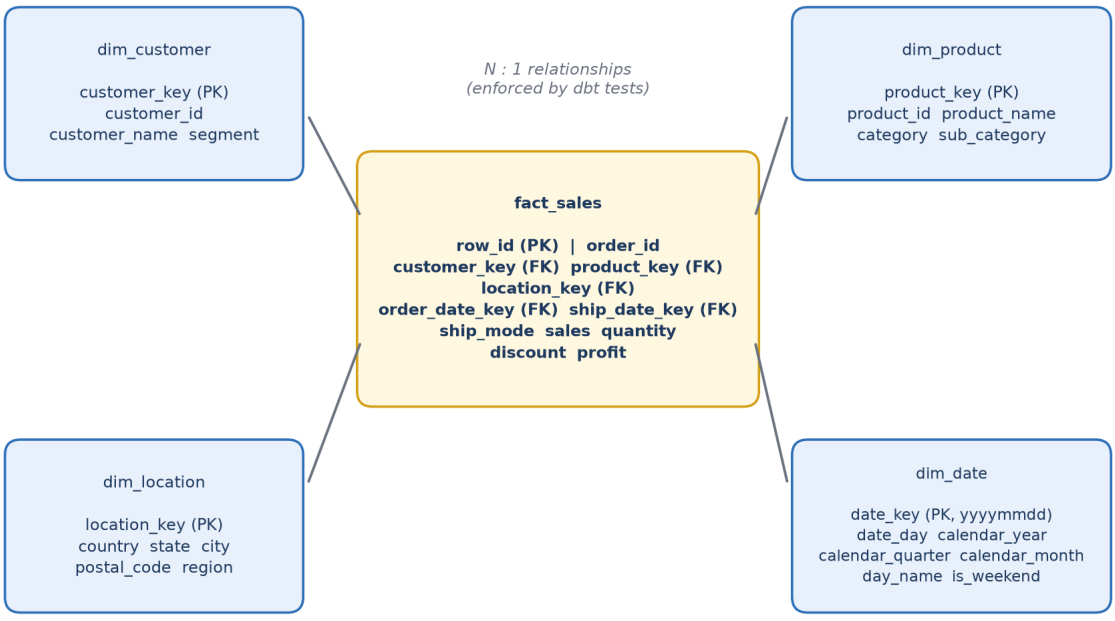


Figure 5: Star schema of the gold layer

6.2 Data dictionary

Table 7: bronze.raw_superstore (metadata columns; the other 21 columns mirror the source file in snake_case)

Column	Type	Description
_batch_id	VARCHAR	Airflow run id of the load that inserted the row.
_loaded_at	TIMESTAMP	UTC time the row was loaded; drives the incremental fact table.
_source_file	VARCHAR	Name of the Excel file.

Table 8: gold.dim_customer

Column	Type	Description
customer_key	VARCHAR	Surrogate key, md5(customer_id). Primary key.
customer_id	VARCHAR	Natural key from the source, for example CG-12520.
customer_name	VARCHAR	Customer name.
segment	VARCHAR	Consumer, Corporate or Home Office.

Table 9: gold.dim_product

Column	Type	Description
product_key	VARCHAR	Surrogate key, md5(product_id). Primary key.
product_id	VARCHAR	Natural key, for example FUR-BO-10001798.
product_name	VARCHAR	Latest product name for the id.
category	VARCHAR	Furniture, Office Supplies or Technology.
sub_category	VARCHAR	Sub-category such as Bookcases or Phones.

Table 10: gold.dim_location

Column	Type	Description
location_key	VARCHAR	Surrogate key, md5 of country, state, city and postal code. Primary key.
country, state, city	VARCHAR	Address levels.
postal_code	VARCHAR	Five-digit postal code kept as text.
region	VARCHAR	Central, East, South or West.

Table 11: gold.dim_date

Column	Type	Description
date_key	INTEGER	Date as yyyymmdd. Primary key.
date_day	DATE	Calendar date.
calendar_year / quarter / month	BIGINT	Date parts.
month_name, day_name	VARCHAR	Names for reporting.
day_of_month, day_of_week	BIGINT	Day of month; day of week (0 = Sunday).
is_weekend	BOOLEAN	True for Saturday and Sunday.

Table 12: gold.fact_sales (grain: one order line)

Column	Type	Description
row_id	INTEGER	Source Row ID. Primary key and incremental unique key.
order_id	VARCHAR	Order number; several lines can share it.
customer_key, product_key, location_key	VARCHAR	Foreign keys to the dimensions.
order_date_key, ship_date_key	INTEGER	Foreign keys to dim_date.
ship_mode	VARCHAR	Same Day, First Class, Second Class or Standard Class.
sales	DOUBLE	Revenue of the line in USD.
quantity	INTEGER	Units sold.
discount	DOUBLE	Discount rate between 0 and 1.
profit	DOUBLE	Profit in USD; negative when the line lost money.
_batch_id, _loaded_at	VARCHAR / TIMESTAMP	Lineage columns copied from silver.

Table 13: marts.* (columns of the four marts)

Mart	Columns	Notes
mart_monthly_sales	month_start, total_sales, total_profit, order_count, units_sold	One row per month (48).
mart_category_region_profit	category, sub_category, region, total_sales, total_profit, profit_margin	Margin = profit / sales.
mart_customer_rfm	customer_key, customer_name, last_order_date, recency_days, frequency, monetary, r_score, f_score, m_score, rfm_segment	One row per customer.
mart_shipping_performance	ship_mode, region, order_lines, avg_days_to_ship, max_days_to_ship	16 rows.

Table 14: ops.pipeline_run_log

Column	Type	Description
batch_id	VARCHAR	Airflow run id.
status	VARCHAR	success or failed.
rows_loaded, rows_rejected	INTEGER	Counts for this batch only.
duration_seconds	DOUBLE	Time from the start of the run to the log entry.
failed_task	VARCHAR	Task that failed (null on success).
finished_at	TIMESTAMP	UTC time of the log entry.

7. Testing and Data Quality

7.1 Approach

Code was written test-first wherever it is Python: a failing test was written, run to see it fail for the right reason, and only then was the code written to make it pass. The same discipline was used for the silver reject rules through a dbt unit test. Bugs that appeared later were first captured by a failing test and then fixed. The checks fall into three groups.

Table 15: Automated checks

Group	What is checked	Where
Python unit tests (pytest)	Loader: first load, re-run adds nothing, extra rows append, leading-zero postal codes, Excel serial dates, text in a date column, missing file, wrong sheet, missing column, duplicate row_id, header-only file. Ops script: reject-rate logic, per-batch gate, run logging, failure logging before any table exists.	tests/test_load_bronze.py, tests/test_pipeline_ops.py
DAG tests (inside the Airflow image)	Every task keeps its own id, the tasks run in the documented order, bronze tests do not pull in later-layer checks, the gate judges the current batch.	tests/test_dag.py
dbt data tests	Not-null and unique keys, accepted values for ship mode and RFM segment, relationships from the fact table to all four dimensions, and reconciliation tests (bronze = silver + rejects; fact rows and total sales equal silver).	models/*/schema.yml, dbt_project/tests/
dbt unit tests	Silver: five artificial rows must get the exact reject reason. Marts: customers with tied RFM values must get stable, key-ordered scores.	models/silver/unit_tests.yml, models/marts/unit_tests.yml

7.2 Results

Table 16: Test results at the end of the project

Suite	Result
pytest on the host (loader and ops script)	22 passed, 1 skipped (the DAG tests need Airflow)
pytest inside the Airflow image (DAG tests)	4 passed
dbt build (models, snapshots, tests)	65 of 65 nodes passed
End-to-end Airflow run on an empty warehouse	success, 9,994 rows loaded, 0 rejected
Second run with the same file	success, 0 rows loaded, fact table unchanged
Run with the source file removed	failed after retries; logged with failed_task = check_source_file

7.3 Why reconciliation tests matter

Unique and not-null tests check individual tables, but they cannot show that rows went missing between layers. Two singular tests close that gap: bronze rows must equal silver rows plus rejected rows, and the fact table must have the same number of rows and the same total sales as silver. A join that silently drops rows, for example a missing dimension key, makes these tests fail.

8. Deployment and Operation

8.1 Prerequisites

- Docker Desktop with the WSL2 backend (Windows) or Docker Engine with the Compose plugin.
- The file data/raw/Superstore.xlsx (included in the repository).

8.2 Start the stack

```
git clone https://github.com/omarmohosny/superstore-pipeline.git
cd superstore-pipeline
docker compose -f airflow/docker-compose.yaml up -d --build
# Airflow UI: http://localhost:8080  (no login in this local setup)
```

In the UI, unpause `superstore_pipeline` and trigger a run. A full run takes about one minute.

8.3 Useful commands

```
# look at any table
bash scripts/in_container.sh '$PIPELINE_BIN/python $PROJECT_DIR/scripts/query.py "select count(*) from gold.fact_sales"'

# run every dbt model and test by hand
bash scripts/in_container.sh '$PIPELINE_BIN/dbt build'

# host tests (loader and ops script)
python -m pytest
```

8.4 Loading corrected data

Bronze is append-only by `row_id`. If a row that is already loaded is corrected in the Excel file, delete it from bronze first so the next run loads it again: `delete from bronze.raw_superstore where row_id in (...)`.

9. Results

9.1 Business results from the marts

The pipeline produced $2,297,201 in total sales and $286,397 in total profit (12.5% margin) across 5,009 orders. Figures 6 to 8 are drawn directly from the mart tables.

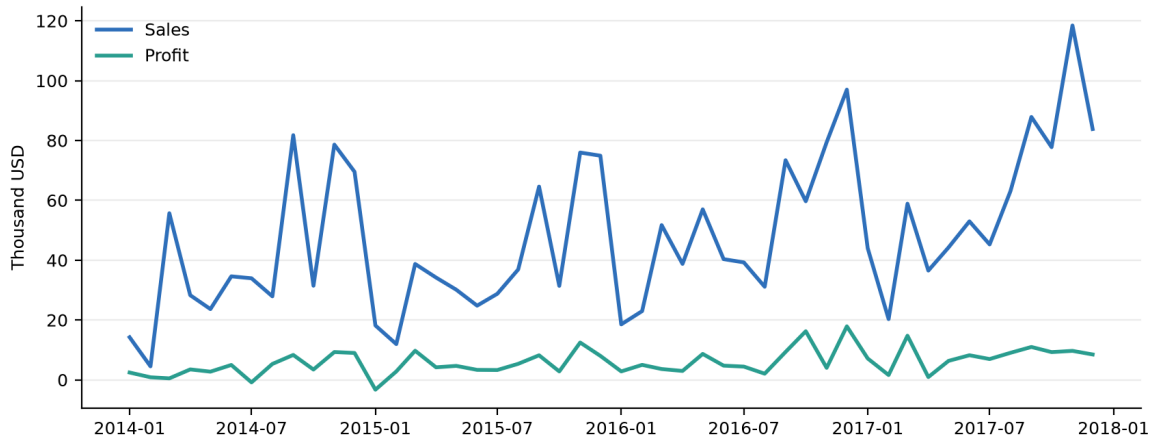


Figure 6: Monthly sales and profit, 2014 to 2017 (mart_monthly_sales)

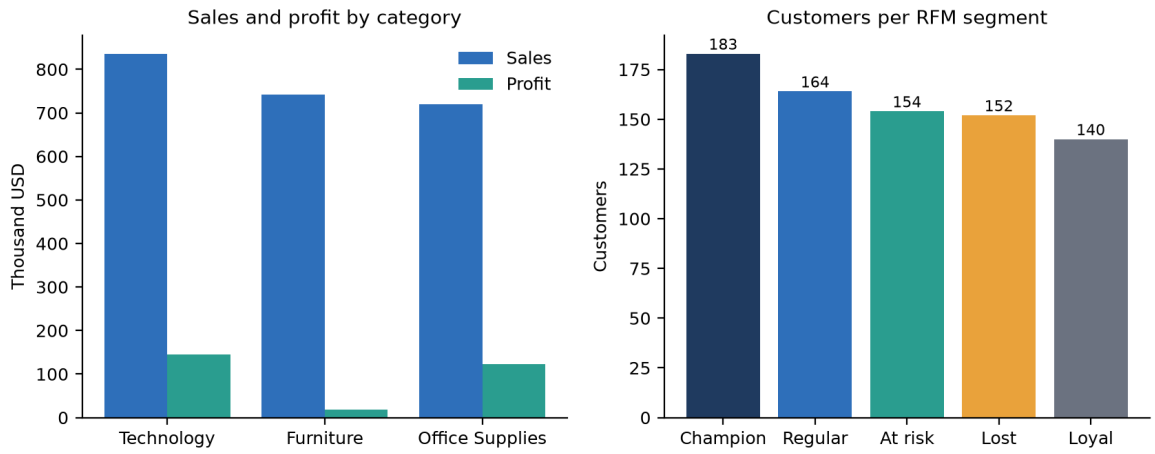


Figure 7: Sales and profit by category, and customers per RFM segment

Technology is the most profitable category. The most profitable sub-categories are Copiers ($55,618), Phones ($44,516), Accessories ($41,937). The weakest are Tables (-$17,725), Bookcases (-$3,473), Supplies (-$1,189); negative values mean those sub-categories lose money overall, which is a useful finding for discount policy. Sales follow a clear seasonal pattern with peaks in September, November and December, and sales grow from 2015 to 2017.

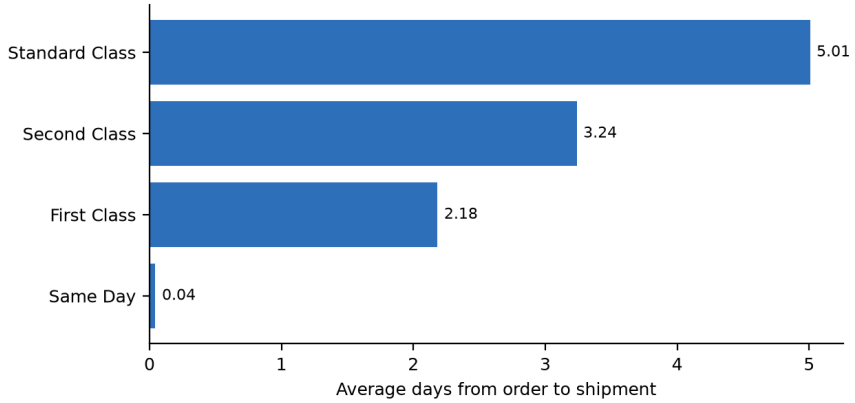


Figure 8: Average days from order to shipment by ship mode

9.2 Pipeline evidence

The most recent logged run has status **success**, loaded 9,994 rows, rejected 0 and took about 46 seconds. The following screenshots come from the running system.

Screenshot not yet added
Airflow UI, DAGs, superstore_pipeline, Graph view of a successful run.
Expected file: docs/screenshots/airflow_graph.png

Figure 9: Airflow graph view of the DAG with all tasks successful

Screenshot not yet added
Airflow UI, superstore_pipeline, Runs tab.
Expected file: docs/screenshots/airflow_runs.png

Figure 10: Airflow run history with successful and failed runs

Screenshot not yet added
Airflow UI, open task test_silver, Logs tab (shows the reject-rate line).
Expected file: docs/screenshots/airflow_gate_log.png

Figure 11: Log of the test_silver quality gate

Screenshot not yet added
Docker Desktop, Containers.
Expected file: docs/screenshots/docker_containers.png

Figure 12: The five containers of the stack in Docker Desktop

Screenshot not yet added
dbt docs, lineage graph (optional).
Expected file: docs/screenshots/dbt_lineage.png

Figure 13: dbt lineage graph from the generated documentation

10. Issues Found and Fixed

Running the system end to end, reviewing the code and preparing this document found several defects. Each one was captured by a test that failed before the fix and passes after it.

Table 17: Defects and their fixes

#	Defect	How it was found	Fix and regression test
1	Task ids became check_source_file__1, __2, ... because one shared decorator object was reused for every task.	First end-to-end run in Airflow.	Create a fresh decorator for each task. test_every_task_keeps_its_own_id.
2	test_bronze also ran a reconciliation test that depends on silver, which did not exist yet on an empty warehouse.	First end-to-end run (task failed).	Add --indirect-selection cautious. test_bronze_tests_do_not_pull_in_silver_checks.
3	Host pytest tried to import the project's own airflow/ folder as the Airflow package.	Running the suite on the host.	Skip on airflow.models instead of airflow.
4	449 postal codes lost their leading zero (06824 became 6824) because Excel stores them as numbers.	Code review, confirmed on the real file.	Pad numeric codes to five digits. test_numeric_postal_code_gets_its_leading_zero_back.
5	One text value in a date column would have broken the silver build and later loads.	Code review.	Convert dates cell by cell; unreadable values become null and are rejected. test_text_in_a_date_column_becomes_null_and_later_loads_still_work.
6	The reject gate used lifetime totals, so old rejects counted forever and new bad rows could hide in a large history.	Code review.	Judge only the current batch. test_old_rejects_do_not_count_against_a_new_batch, test_reject_gate_judges_the_current_batch.
7	A test for text dates passed too early because pandas reads the string N/A as an empty value.	Watching the test fail before the fix.	Use a different text value (TBD) so the test fails for the right reason.
8	RFM segment counts changed between two runs on identical data, because quartile scoring (ntile) gave tied customers an arbitrary order.	Comparing two runs while preparing this document.	Add customer_key as a tie-breaker to all three scores. Unit test rfm_scores_break_ties_by_customer_key.

Known limitations

- Gold dimensions read the current silver tables, not the snapshots, so the SCD2 history is stored but not yet used by reports.
- The snapshot task runs before the quality gate, so history from a batch that the gate later rejects is kept.
- `fact_sales` never deletes rows. If a reject rule becomes stricter, run dbt with `--full-refresh`.
- DuckDB has one writer: reading the database while a DAG run is writing can fail that task; the retries normally cover it.
- The Compose file has no user mapping, so the mounted warehouse folder may need permissions on Linux hosts.

11. Future Work

- Add a dashboard (for example Streamlit or Metabase) on top of the marts.

- Send failure alerts by e-mail or chat from the existing callback function.
- Use the snapshots in the dimensions to support point-in-time reporting.
- Add CI that runs the pytest suite and dbt build on every push.
- Move the warehouse to a server database if several writers are needed.

12. Conclusion

The project delivers a complete, tested ELT pipeline: an Excel file becomes a validated star schema and four analytics tables, built on a schedule, with a quality gate that stops bad data and a log that records every run. Writing the Python parts test-first, adding reconciliation tests between layers, and reviewing the finished system caught real defects early. The Docker setup lets anyone reproduce the result with a single command.

Appendix A: Key Source Code

A.1 Reject rules (dbt model silver_checked)

```
select
    *,
    case
        when row_id is null or order_id is null
            or customer_id is null or product_id is null         then 'missing key'
        when order_date is null or ship_date is null             then 'missing date'
        when sales is null or sales <= 0                         then 'non-positive sales'
        when ship_date < order_date                              then 'ship date before order date'
        when discount < 0 or discount > 1                        then 'discount out of range'
    end as reject_reason
from typed
```

A.2 Incremental fact table (dbt model fact_sales)

```
{{ config(materialized='incremental', unique_key='row_id') }}

select
    s.row_id,
    s.order_id,
    c.customer_key,
    p.product_key,
    l.location_key,
    cast(strftime(s.order_date, '%Y%m%d') as integer) as order_date_key,
    cast(strftime(s.ship_date, '%Y%m%d') as integer)  as ship_date_key,
    s.ship_mode,
    s.sales,
    s.quantity,
    s.discount,
    s.profit,
    s._batch_id,
    s._loaded_at
from {{ ref('silver_superstore') }} s
join {{ ref('dim_customer') }} c on c.customer_id = s.customer_id
join {{ ref('dim_product') }} p on p.product_id = s.product_id
join {{ ref('dim_location') }} l on l.location_key = {{ location_key('s') }}

{% if is_incremental() %}
where s._loaded_at > (select max(_loaded_at) from {{ this }})
{% endif %}
```

A.3 Bronze loader core (scripts/load_bronze.py)

```
def to_date(col):
    if pd.api.types.is_datetime64_any_dtype(col):
        return col
    # Excel serial numbers where the cell is a number, plain text parsing otherwise;
    # anything unreadable becomes null and gets rejected in silver
    serial = pd.to_numeric(col, errors="coerce")
    from_serial = pd.to_datetime(serial, unit="D", origin="1899-12-30")
    from_text = pd.to_datetime(col.where(serial.isna()), errors="coerce")
    return from_serial.fillna(from_text)


def fix_dates(df):
    for col in DATE_COLUMNS:
        df[col] = to_date(df[col])
    return df


def fix_postal_codes(df):
    # Excel stores them as numbers, so 06824 arrives as 6824
    codes = df["postal_code"].str.strip()
    df["postal_code"] = codes.mask(codes.str.fullmatch(r"\d{1,4}").fillna(False), codes.str.zfill(5))
    return df


def check_rows(df):
    if df.empty:
        raise ValueError("source file has no rows")
    if df["row_id"].isna().any():
        raise ValueError("source file has rows without a row_id")
    dupes = df.loc[df["row_id"].duplicated(), "row_id"].unique().tolist()
    if dupes:
        raise ValueError(f"duplicate row_id in source file: {dupes[:5]}")
```

```
def table_exists(con):
    schema, table = TABLE.split(".")
    found = con.execute(
        "select count(*) from information_schema.tables where table_schema = ? and table_name = ?",
        [schema, table],
    ).fetchone()[0]
    return found > 0


def count_rows(con):
    return con.execute(f"select count(*) from {TABLE}").fetchone()[0]


def load_bronze(xlsx_path, db_path, batch_id):
    df = fix_postal_codes(fix_dates(read_source(xlsx_path)))
    check_rows(df)

    df["_batch_id"] = batch_id
    df["_loaded_at"] = datetime.now(timezone.utc).replace(tzinfo=None)
    df["_source_file"] = Path(xlsx_path).name

    Path(db_path).parent.mkdir(parents=True, exist_ok=True)
    con = duckdb.connect(str(db_path))
    try:
        con.execute("create schema if not exists bronze")
        con.register("incoming", df)

        if not table_exists(con):
            con.execute(f"create table {TABLE} as select * from incoming")
            return len(df)

        before = count_rows(con)
        con.execute(
            f"""
            insert into {TABLE} by name
            select * from incoming i
            where not exists (select 1 from {TABLE} r where r.row_id = i.row_id)
            """
        )
        return count_rows(con) - before
    finally:
        con.close()
```

A.4 DAG definition (airflow/dags/superstore_pipeline.py)

```
TASK_ENV = {
    "BATCH_ID": "{{ run_id }}",
    "STARTED_AT": "{{ dag_run.start_date.isoformat() }}",
    "REJECT_THRESHOLD": "{{ var.value.get('reject_rate_threshold', '0.02') }}",
}


def shell():
    # a fresh decorator per task, so every task keeps its own task_id
    return task.bash(env=TASK_ENV, append_env=True)


def log_failure(context):
    # runs for the task that failed, even if bronze or silver were never built
    subprocess.run(
        [
            os.path.join(os.environ["PIPELINE_BIN"], "python"),
            os.path.join(os.environ["PROJECT_DIR"], "scripts", "pipeline_ops.py"),
            "log",
            "--db", os.environ["SUPERSTORE_DB"],
            "--batch-id", context["run_id"],
            "--status", "failed",
            "--started-at", context["dag_run"].start_date.isoformat(),
            "--failed-task", context["ti"].task_id,
        ],
        check=False,
    )


@dag(
    dag_id="superstore_pipeline",
    schedule="@daily",
    start_date=datetime(2026, 1, 1),
    catchup=False,
    max_active_runs=1,
    default_args={
        "retries": 2,
```

```
        "retry_delay": timedelta(minutes=1),
        "on_failure_callback": log_failure,
    },
    tags=["superstore", "dbt"],
)
def superstore_pipeline():

    @shell()
    def check_source_file():
        return 'test -f "$SUPERSTORE_FILE" || { echo "source file missing: $SUPERSTORE_FILE"; exit 1; }'

    @shell()
    def dbt_debug():
        return "$PIPELINE_BIN/dbt debug"

    @shell()
    def load_bronze():
        return (
            "$PIPELINE_BIN/python $PROJECT_DIR/scripts/load_bronze.py "
            '--file "$SUPERSTORE_FILE" --db "$SUPERSTORE_DB" --batch-id "$BATCH_ID"'
        )

    @shell()
    def test_bronze():
        return "$PIPELINE_BIN/dbt test --select source:bronze --indirect-selection cautious"

    @shell()
    def build_silver():
        return "$PIPELINE_BIN/dbt run --select silver"

    @shell()
    def snapshot():
        return "$PIPELINE_BIN/dbt snapshot"

    @shell()
    def test_silver():
        # gate: dbt tests first, then the reject-rate limit
        return (
            "$PIPELINE_BIN/dbt test --select silver tag:silver_reconciliation --indirect-selection cautious"
            " && $PIPELINE_BIN/python $PROJECT_DIR/scripts/pipeline_ops.py check-rejects"
            ' --db "$SUPERSTORE_DB" --batch-id "$BATCH_ID" --threshold "$REJECT_THRESHOLD"'
        )

    @shell()
    def build_gold():
        return "$PIPELINE_BIN/dbt run --select gold marts"

    @shell()
    def test_gold():
        return (
            "$PIPELINE_BIN/dbt test --select gold marts tag:reconciliation --indirect-selection cautious"
            " && $PIPELINE_BIN/dbt docs generate"
        )

    @shell()
    def log_run():
        return (
            "$PIPELINE_BIN/python $PROJECT_DIR/scripts/pipeline_ops.py log "
            '--db "$SUPERSTORE_DB" --batch-id "$BATCH_ID" --status success --started-at "$STARTED_AT"'
        )

    (
        check_source_file()
        >> dbt_debug()
        >> load_bronze()
        >> test_bronze()
        >> build_silver()
        >> snapshot()
        >> test_silver()
        >> build_gold()
        >> test_gold()
        >> log_run()
    )


superstore_pipeline()
```